

Designing a GitOps-Native Kubernetes Platform as a Product

Building a golden path that's fast, secure, and consistent — by design, not by heroics.



Goutham K

SWE II @ Akamai, building Akamai Cloud



What We'll Cover

I

Why Platform Engineering

DevOps's autonomy tax · ticket ops vs. self-service · the golden path

II

The GitOps Spine

GitOps as the backbone · management cluster to app clusters

III

Building the Stack, Tool by Tool

ArgoCD · Crossplane · Kyverno · Cilium · and more

IV

The Full Picture

Shared responsibility · success metrics · a day in the life

V

Key Takeaways

What worked, what we'd do differently, and what's next

DevOps gave teams freedom. Freedom has a tax.



Autonomy accelerated delivery

Independent teams shipped faster — but inherited infrastructure they never signed up to own.



Cognitive load went up, not down

"Build the feature" quietly became "build the feature, and the pipeline, and the cluster config."



Duplicated effort, duplicated risk

Every team solving the same infrastructure problems differently means every team can fail differently, too.

Ticket Ops vs. Self-Service



Ticket Ops

- File a request, then wait
- Manual, one-off provisioning
- Knowledge lives in someone's head
- Every request queues behind the last one



Self-Service

- Request and receive, autonomously
- Automated, golden-path provisioning
- Knowledge lives in docs and templates
- Bottlenecked by nothing but your own backlog

Platform Engineering

Building internal developer platforms that give teams self-service over their full delivery lifecycle — from code to production.



Developer Self-Service

Build, test, deploy, and roll back — entirely through self-service, without opening a ticket.



Reduce Cognitive Load

Platform teams absorb infrastructure complexity so product teams can stay focused on features.



Platform as a Product

Internal developers are your customers. Their adoption — not a mandate — is what makes it succeed.



Treat the Platform Like a Product



Internal developers are your customers

Their adoption, not a leadership mandate, is what makes the platform succeed.



Roadmap driven by feedback, not decree

Built with the teams who'll use it, prioritized by what actually unblocks them.



Thin and opinionated, not all-encompassing

The platform is the smallest layer that gives a consistent experience — not a rebuild of every tool underneath it.

The Data Backs This Up

DORA's 2025 State of AI-Assisted Software Development report — nearly 5,000 respondents

90%

of organizations now run an
internal developer platform

76%

have a dedicated platform team

5%

productivity gain from developer
independence alone

DORA'S CORE FINDING

AI doesn't fix a fragmented platform — it amplifies whatever's already there. When platform quality is high, AI's effect on performance is strong and positive. When it's low, the effect is negligible.



The Golden Path

A paved, opinionated route from idea to production — pre-wired with the right defaults, so the right way is also the easy way.

So what does a golden path actually look like... on Kubernetes?

Kubernetes makes this easier and harder



Declarative APIs are perfect for self-service

Custom resources are a natural fit for "describe what you want" platform interfaces.



...but CRDs, controllers, and multi-tenancy multiply what can go wrong

More moving parts, more failure modes, more ways for one team's mistake to reach another's.



For security-sensitive workloads, speed and safety can't trade off

Regulated and security-sensitive teams need both at once — not one at the expense of the other.

Three Principles, One Platform



GitOps

Git is the single source of truth for everything — apps and infrastructure alike.



Zero Trust

Least privilege by default. Every escalation is a reviewed, logged change.



Shift-Left Governance

Guardrails are built into the pipeline from day one — not bolted on after an audit.

GitOps, Concretely



Desired state lives in Git

Not in someone's terminal history, and not in a console click someone forgot to document.



A controller continuously reconciles reality to match it

The cluster is always being pulled back toward what Git says it should be.



Every change is a pull request

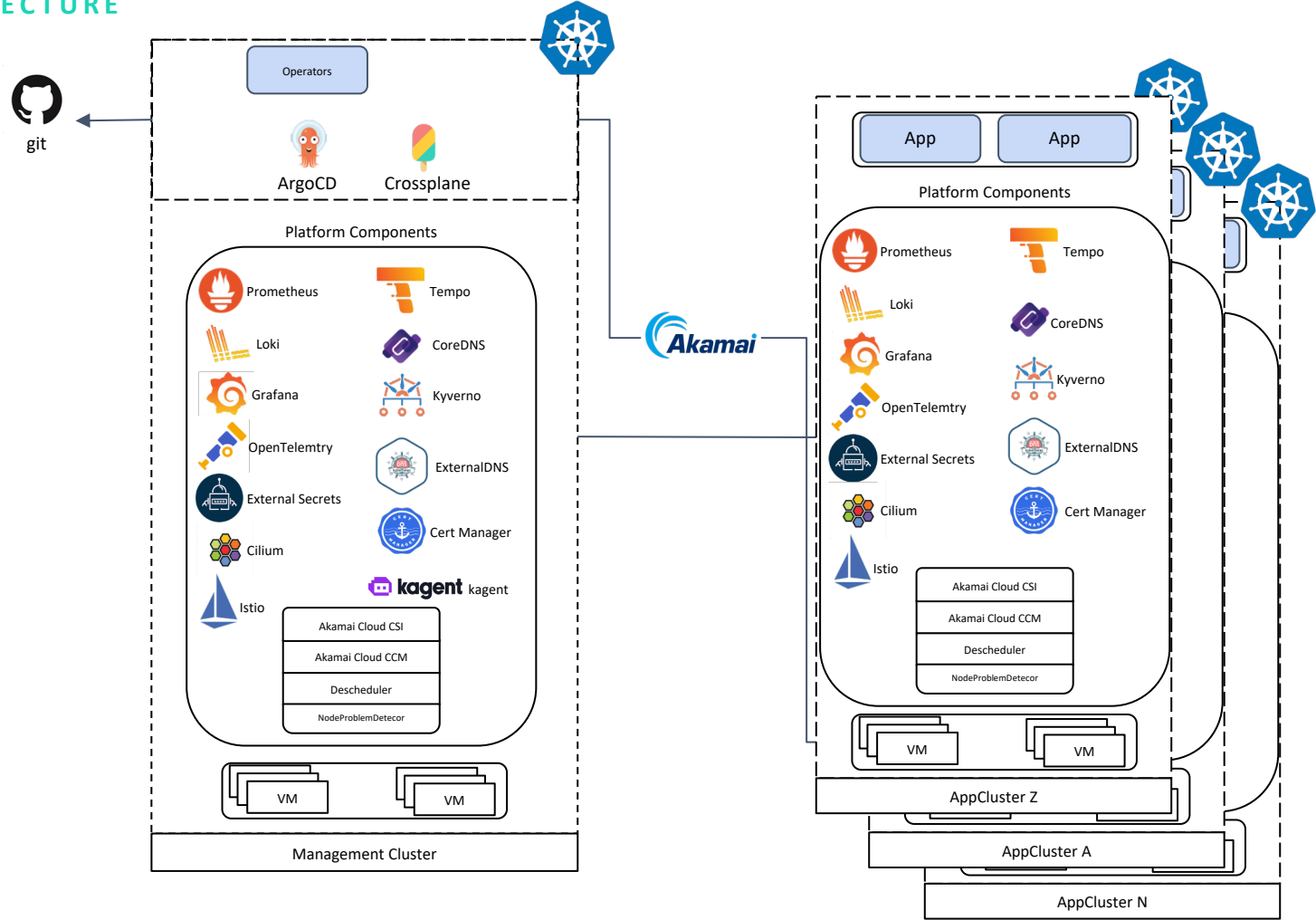
Reviewed, approved, and logged — the exact same workflow your team already trusts for code.



Drift isn't a fire drill

Manual changes get auto-corrected, and rollbacks are just reverting a commit.

ACT 2 · THE ARCHITECTURE



Clusters That Arrive Pre-Wired



Hardened base image, secure by default

CIS-aligned baseline configuration from the moment the cluster exists — not bolted on afterward.



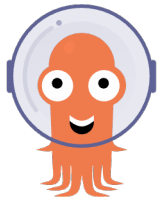
Baseline add-ons, already installed

Observability, certificate management, and vulnerability scanning ship with every cluster.



Teams version against the platform, not a dozen tools

One version number to track, not a matrix of independent component versions.



ACT 3 · BUILDING IT LAYER BY LAYER

ArgoCD: the Platform's Control Plane



App-of-apps: one repo describes the whole platform

A single Git tree is the entry point for every cluster, every add-on, every config.



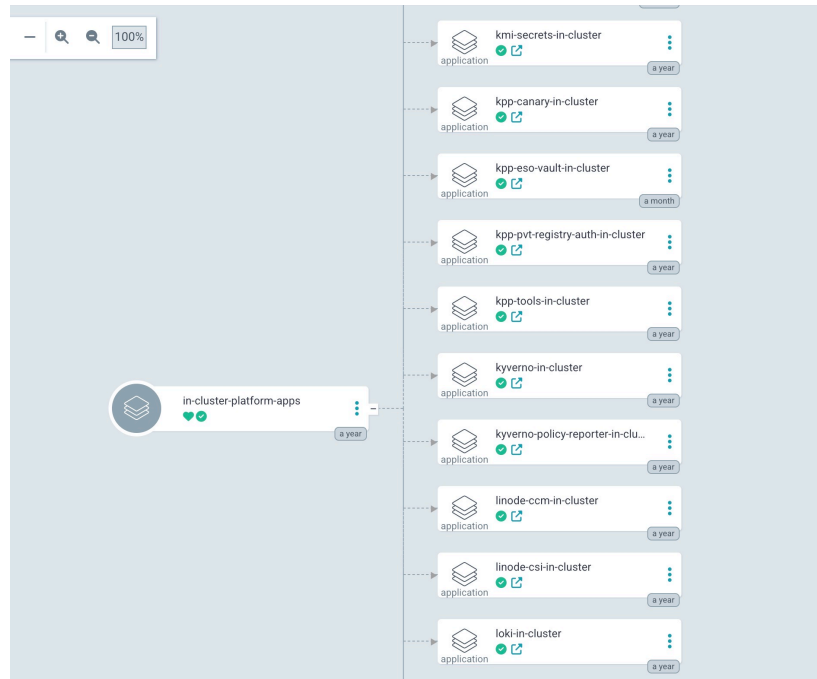
Continuous sync between Git and cluster state

What's running is always converging toward what's declared — automatically, on a loop.



One auditable pane of glass

Every change, who approved it, and when it shipped — all in one place, all queryable.





ACT 3 · BUILDING IT LAYER BY LAYER

Crossplane: Infrastructure as an API



"Give me a cluster" becomes a custom resource

Provisioning infrastructure looks exactly like provisioning any other Kubernetes object.



Compositions are the golden path, codified

The platform team defines what "a cluster" means once; every team gets the same good defaults.



Management policies prevent drift and accidental deletion

Manual changes get reconciled back; destructive actions need to go through Git, not a console.



Kyverno: Guardrails by Default



A restricted security baseline, everywhere

Every workload, in every namespace, starts from the same locked-down default posture.



No more "please review my YAML"

Policy isn't a person in a review queue — it's enforced automatically, the same way every time.



Caught at admission, not in production

A misconfigured workload is rejected before it ever starts running, not discovered after.

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  annotations:
    policies.kyverno.io/category: Security
    policies.kyverno.io/description: This policy prevents deletion of critical Crossplane
      Provider resources to maintain cluster stability and prevent accidental removal
      of essential infrastructure.
    policies.kyverno.io/severity: high
    policies.kyverno.io/title: Prevent Provider Deletion
  name: prevent-provider-deletion
spec:
  admission: true
  background: true
  emitWarning: false
  rules:
    - match:
        any:
          - resources:
              kinds:
                - Provider
              names:
                - provider-linode
              operations:
                - DELETE
      name: prevent-linode-provider-deletion
      skipBackgroundRequests: true
      validate:
        allowExistingViolations: true
        deny: {}
        message: Deletion of the provider-linode Provider resource is not allowed. This
          resource is critical for cluster operations.
      validationFailureAction: enforce
```

...With Room to Breathe



The exception pattern

- Real workloads occasionally need a narrow exception — say, one sidecar that genuinely needs elevated privilege.
- The exception is scoped to that one case, written declaratively, and lives in Git — not in a Slack thread or someone's memory.
- At audit time, the exception *is* the documentation: what was allowed, for what, approved by whom, since when.

Cilium: Zero Trust at the Network Layer



eBPF dataplane

Policy enforcement in the kernel — no sidecar proxy required for basic L3/L4 control.



L3 through L7 network policy

Control who can talk to whom, down to the specific API call or HTTP method.



Deny by default, allow deliberately

Connectivity is something you grant in Git, not something that exists until proven dangerous.

Filling the Gap: a Service Mesh for Encryption



Cilium handles policy; Istio adds transparent mTLS

Pod-to-pod traffic gets encrypted in transit without any application code change.



Used together, not as competitors

Each covers a layer the other doesn't — network policy and identity-based encryption aren't the same problem.

Audit Everything, Alert on the Exceptions



Every API action is logged, tied to a real identity

Not a shared service account — an individual, traceable user or system.



Privileged credentials are expected to come from one place

The GitOps controller is the only "normal" user of high-privilege access.



Anyone else using them is an alert, not a surprise

Deviation from that expectation is exactly the signal worth investigating first.

Onboarding Without a Ticket



OIDC makes it easy

Same client ID, no secrets



The same SSO config ships to every team

Nothing bespoke to configure per cluster — onboarding doesn't require a ticket to an identity team.



Same-day access in, clean access out

Offboarding is exactly as simple as onboarding, because identity was never duplicated anywhere.

RBAC That Doesn't Require a PhD



A simplified config maps teams to roles

Most teams never have to write a line of raw Kubernetes RBAC YAML.



Power users can still drop to native RBAC

The simple format expands into real ClusterRoleBindings — nothing is hidden or limited.



Either way, it's a reviewed pull request

Code owners and CI checks apply to permissions exactly like they apply to application code.

Upgrades Become Git Diffs



The platform ships as one versioned bundle

Cluster version and every add-on version travel together — no mix-and-match matrix to test.



Teams adopt a new version on their own schedule

The platform team ships it; each team chooses when they're ready to move.



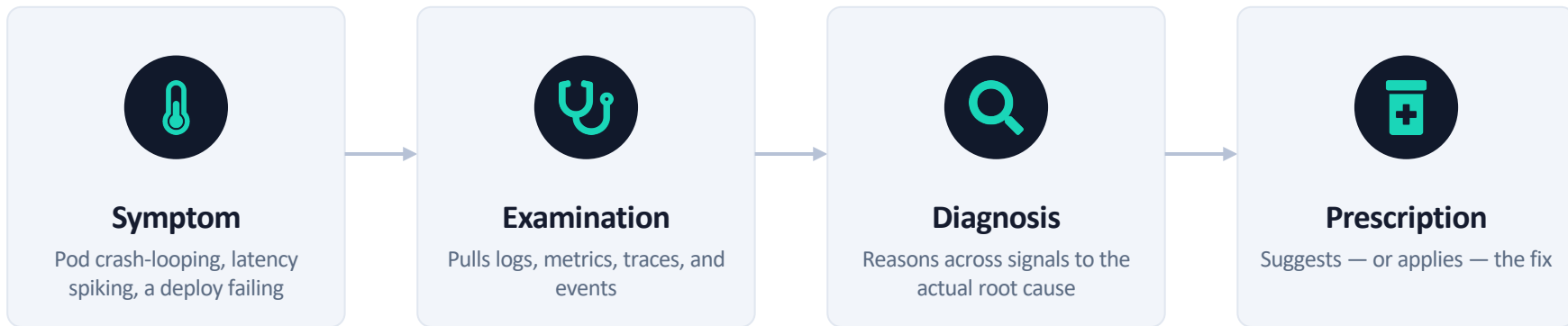
Rolling back is reverting a commit

Not a 2am incident — just the same Git workflow, run in reverse.

BONUS · CLOSING THE LOOP

kagent: A Doctor for Your Cluster

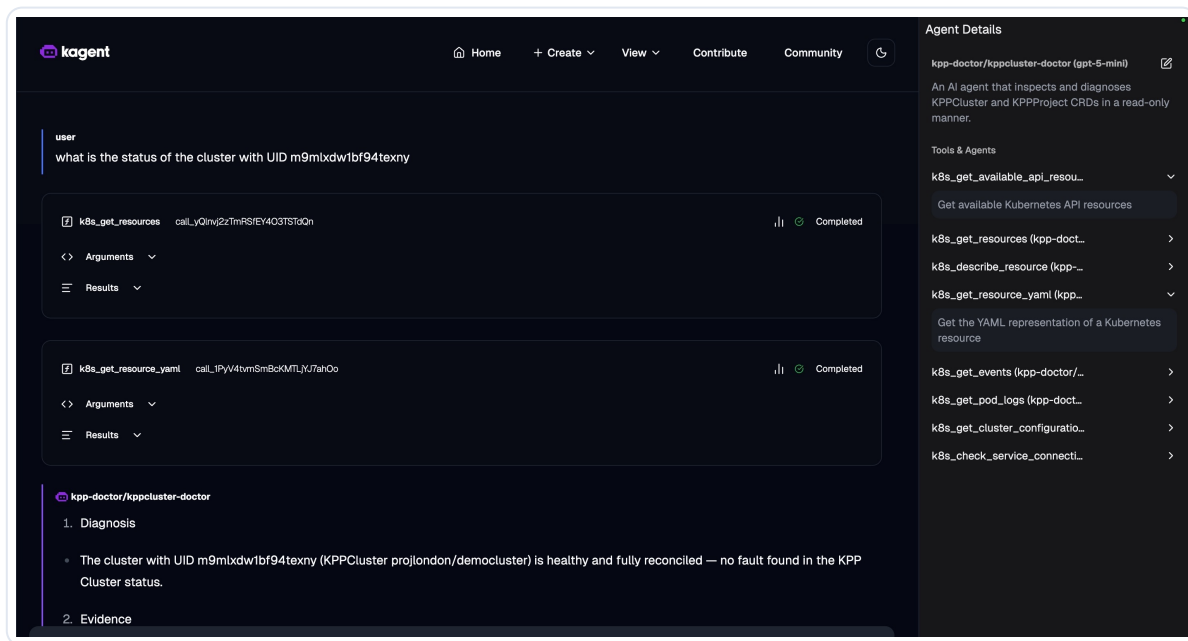
An open-source, Kubernetes-native AI agent (CNCF Sandbox) that debugs the way a clinician would — symptom, exam, diagnosis, fix.



BONUS · CLOSING THE LOOP

Not Just a Metaphor

A real kagent agent — diagnosing a live cluster, read-only.



Real tool calls → real diagnosis: “healthy and fully reconciled — no fault found.”

A Shared Responsibility Model



The Platform Team Owns

- The control plane and its reliability
- The golden path and its defaults
- The guardrails everyone inherits
- Common tools every team can reuse



Every Team Owns

- How they actually use the platform
- Their own workload's security posture
- Their own compliance story end-to-end
- Choosing when they adopt a new version

Measuring Whether It's Working



Deployment frequency & lead time for changes

The classic DORA pair — borrowed because they still work.



Time to first deploy for a brand-new service

How long from "we have an idea" to "it's running in production"?



Time from CVE disclosure to "we know who's affected"

Central visibility turns this from a scramble into a lookup.



Adoption — are teams choosing the platform, or routing around it?

The only metric that tells you if "platform as a product" is actually true.

A Day in the Life



Opens a PR requesting a new cluster

It's merged; the cluster exists a few minutes later. No ticket, no human in the loop.



SSO just works

Access matches their team's role automatically — nothing requested, nothing extra granted.



Ships a change; a guardrail catches a misconfig

Rejected at admission, with a clear reason — before it was ever a production incident.



Never files a ticket, never waits on a human

That sentence is the entire pitch for treating the platform as a product.

Takeaways



Treat the platform as a product, not a project.

Adoption is earned from users, not mandated from leadership.



Make the golden path the path of least resistance.

If the secure way is also the easy way, people will take it.



Guardrails belong in Git, not in someone's memory.

Declarative, reviewable, and still there after that person changes teams.



Security and speed aren't opposites when both are automated.

The trade-off only exists when one of them is still manual.



Thank You

Questions?



github.com/gouthamhusky